

Encrypting Amazon S3 Buckets and Managing Secure Access Using AWS KMS

Submitted By: Suraj Somkuwar

Table Of Content

Encrypt Amazon S3 Buckets and Manage Secure Access Using AWS KMS	1
Table Of Content	2
1. Executive Summary	3
2. Introduction	3
3. Lab Objectives	4
4. AWS Services Used	4
5. Lab Architecture	4
6. Prerequisites	5
7. Practical Implementation	5
Step 1 – Creating the Amazon S3 Bucket	5
Step 2 – Creating a Customer-Managed AWS KMS Key	6
Step 3 – Configuring Default Server-Side Encryption Using AWS KMS (SSE-KMS)	7
Step 4 – Creating a Custom IAM Policy for Authorized Access	8
Step 5 – Attaching the Custom IAM Policy to Alice	10
Step 6 – Enabling AWS Management Console Access	11
Step 7 – Configuring the Amazon S3 Bucket Policy	12
Step 8 – Validating Authorized User (Alice) Access	14
Step 8.1 – Alice's First Login to AWS Console	14
Step 8.2 – Creating Programmatic Access for Alice	15
Step 8.3 – Configuring AWS CLI for Alice	15
Step 8.4 – Uploading an Object to the Encrypted Bucket	16
Step 8.5 – Viewing Uploaded Objects in AWS Console	17
Step 8.6 – Downloading the Encrypted Object	17
Step 9 – Validating Unauthorized User (Bob) Access	18
Step 9.1 – Bob Login Verification	18
Step 9.2 – AWS CLI Configuration for Bob	19
Step 9.3 – Console Access Denied	19
Step 9.4 – Direct Bucket Access Denied	20
Step 9.5 – CLI Bucket Listing Denied	20
Step 9.6 – Upload Denied by AWS KMS	20
Step 9.7 – Download Denied	21
8. Security Validation	21
9. Challenges Encountered and Troubleshooting	21
10. Security Best Practices Implemented	21
11. Learning Outcomes	22
12. Conclusion	22
13. Deliverable: Short Report	22
How KMS Encryption Protects the Data	22
How the Applied Access Control Prevents Unauthorized Access	22
14. References	23

1. Executive Summary

Cloud storage security is one of the most important aspects of modern cloud computing because organizations store sensitive business information, customer records, financial documents, and application data within cloud environments. Protecting this information requires strong encryption, proper identity management, and strict access control mechanisms.

The objective of this laboratory exercise was to implement a secure Amazon Simple Storage Service (Amazon S3) environment by combining multiple AWS security services including Amazon S3, AWS Identity and Access Management (IAM), AWS Key Management Service (AWS KMS), Bucket Policies, IAM Policies, and the AWS Command Line Interface (AWS CLI).

During this practical implementation, an Amazon S3 bucket was created to store objects securely. A customer-managed AWS KMS encryption key was configured and applied as the default encryption mechanism for the bucket using Server-Side Encryption with AWS KMS (SSE-KMS). This ensured that all objects stored within the bucket were automatically encrypted using a dedicated encryption key managed by the AWS account owner.

To demonstrate secure identity-based access control, two IAM users named Alice and Bob were configured. Alice represented an authorized user with carefully controlled permissions to list, upload, and download objects from the encrypted S3 bucket. Bob represented an unauthorized user whose access was intentionally restricted using IAM policies, bucket policies, and AWS KMS permissions.

The AWS CLI was configured separately for both users to validate programmatic access. Practical testing confirmed that Alice could successfully perform authorized operations, while Bob was prevented from accessing encrypted resources because of insufficient IAM permissions and KMS authorization. These results verified that multiple AWS security controls worked together to enforce the principle of least privilege and protect sensitive cloud resources from unauthorized access.

2. Introduction

Amazon Simple Storage Service (Amazon S3) is a highly scalable object storage service offered by Amazon Web Services. Organizations use Amazon S3 to store application data, backups, documents, images, videos, software packages, and other business-critical information. Because this information often contains sensitive or confidential data, it must be protected against unauthorized access and accidental exposure.

AWS provides several security mechanisms to protect data stored within Amazon S3. Identity and Access Management (IAM) controls which users can access AWS resources, while AWS Key Management Service (AWS KMS) provides encryption key management for protecting data at rest. Bucket Policies provide resource-based access control, enabling administrators to define which identities can or cannot access specific buckets or objects.

In this laboratory exercise, multiple AWS security services were combined to implement a layered security architecture. Instead of relying on only IAM permissions, the solution incorporated customer-managed encryption keys, bucket-level encryption, IAM policies, and bucket policies. This defense-in-depth approach ensures that even if one security mechanism is misconfigured, additional layers continue protecting the stored data.

The implementation also included practical validation using both the AWS Management Console and AWS CLI. This allowed verification that authorized users could perform permitted operations while unauthorized users were prevented from accessing protected resources.

3. Lab Objectives

The primary objectives of this laboratory exercise were:

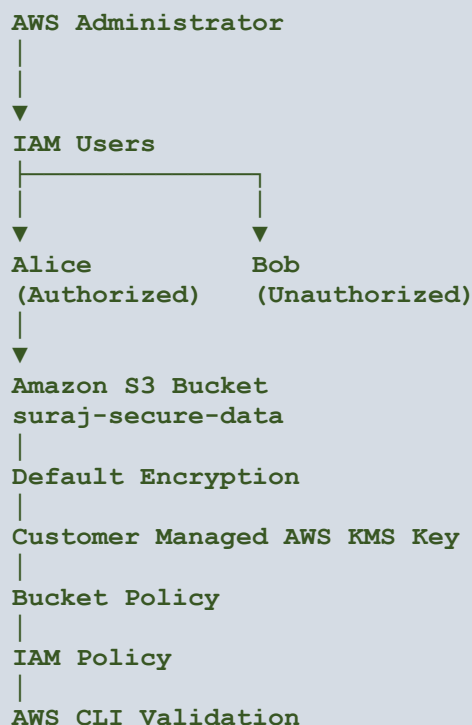
- Create a secure Amazon S3 bucket for object storage.
- Configure a customer-managed AWS KMS encryption key.
- Enable Server-Side Encryption using AWS KMS (SSE-KMS).
- Implement least-privilege IAM policies.
- Create separate IAM users representing authorized and unauthorized users.
- Configure AWS CLI for secure programmatic access.
- Verify successful object upload and download by an authorized user.
- Demonstrate unauthorized access denial using IAM policies, Bucket Policies, and AWS KMS permissions.
- Validate cloud storage security using both the AWS Management Console and AWS CLI.

4. AWS Services Used

AWS Service	Purpose in this Lab
Amazon S3	Secure cloud object storage
AWS IAM	Identity and access management
AWS KMS	Customer-managed encryption keys
Bucket Policy	Resource-level access control
IAM Policy	User-level permission management
AWS CLI	Command-line interaction with AWS services

5. Lab Architecture

The implemented architecture follows the principle of layered security by combining identity management, encryption, and resource-level access control.



The administrator first configured the Amazon S3 bucket and enabled default encryption using a customer-managed AWS KMS key. Two IAM users were then created to demonstrate authorized and unauthorized access

scenarios. IAM policies granted only the required permissions to Alice, while bucket policies and KMS permissions prevented Bob from accessing encrypted objects.

6. Prerequisites

Before beginning the implementation, the following prerequisites were completed:

- AWS Account with administrative privileges.
- AWS CLI Version 2 installed on Windows.
- Internet connectivity.
- AWS Region configured as Asia Pacific (Mumbai) – ap-south-1.
- IAM users created for Alice and Bob.
- Administrative access available to configure IAM, S3, and KMS resources.

7. Practical Implementation

Step 1 – Creating the Amazon S3 Bucket

Objective

The first objective of the implementation was to create a dedicated Amazon S3 bucket that would securely store encrypted objects throughout the laboratory exercise.

Implementation

A new Amazon S3 bucket named suraj-secure-data was created in the Asia Pacific (Mumbai) AWS Region. The bucket served as the primary storage location for all files used during the security validation process. Public access remained blocked to ensure that objects could only be accessed through authenticated AWS identities.

Amazon S3 buckets provide virtually unlimited object storage and form the foundation for many cloud-based applications. Creating a dedicated bucket allowed the implementation of encryption, identity-based access control, and bucket-level security policies without affecting other AWS resources.

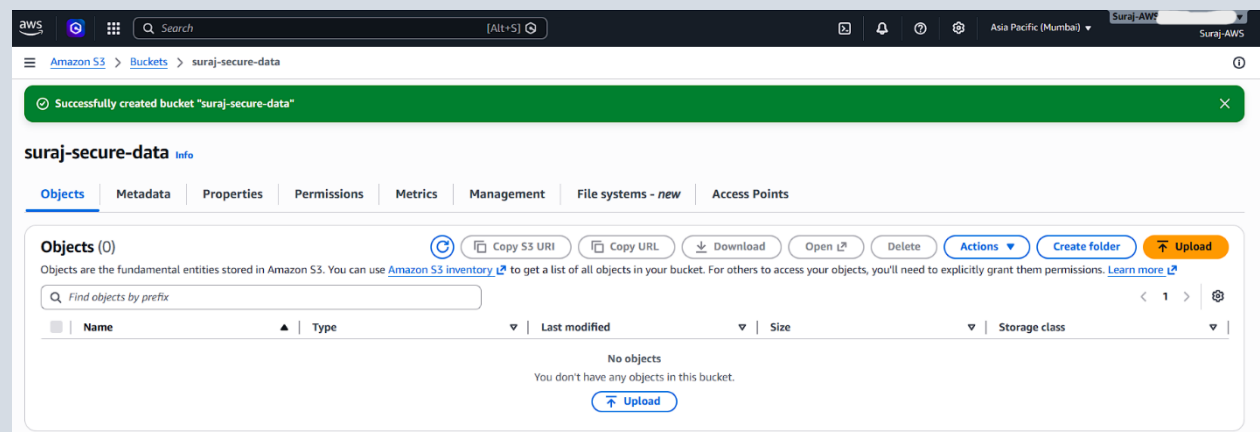


Figure 1. Amazon S3 Bucket Created Successfully

Explanation

The screenshot shows the successful creation of the suraj-secure-data bucket within the Amazon S3 service. The bucket name, AWS Region, and creation details confirm that the storage environment was correctly configured. This bucket serves as the secure repository for encrypted objects used throughout the laboratory.

Security Benefit

Creating a dedicated storage bucket allows security controls such as encryption, bucket policies, and IAM permissions to be applied independently. This isolation follows cloud security best practices by separating sensitive data from unrelated resources.

Result

The Amazon S3 bucket was successfully created and prepared for encryption and access control configuration.

Step 2 – Creating a Customer-Managed AWS KMS Key

Objective

The objective of this step was to create a customer-managed AWS KMS key that would be used to encrypt all objects stored inside the Amazon S3 bucket.

Implementation

AWS Key Management Service (AWS KMS) was used to generate a customer-managed symmetric encryption key. Unlike AWS-managed keys, customer-managed keys provide complete administrative control over key rotation, permissions, and key usage policies.

During the configuration process, the administrator defined the key properties, selected key administrators, and assigned authorized key users. The IAM user Alice was explicitly configured as an authorized key user so that she could successfully encrypt and decrypt objects stored within Amazon S3.

The customer-managed key was later integrated with Amazon S3 using Server-Side Encryption with AWS KMS (SSE-KMS), ensuring automatic encryption of all uploaded objects.

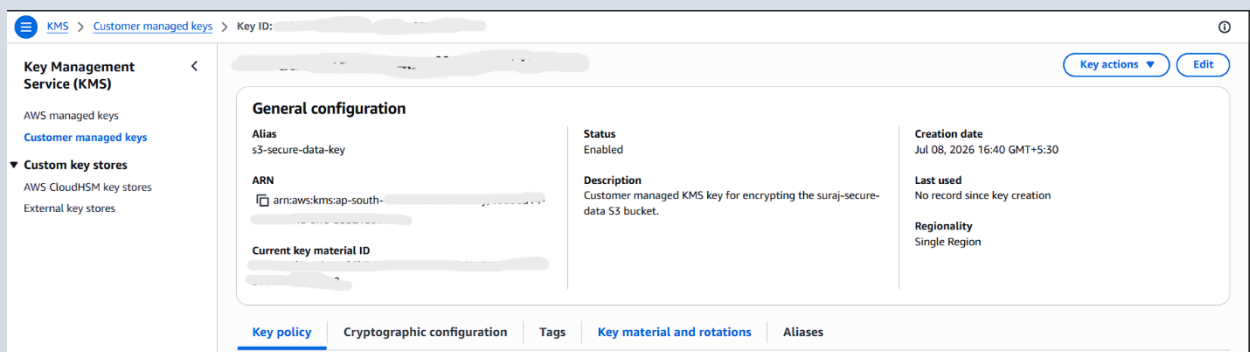


Figure 2. General Configuration of Customer-Managed AWS KMS Key

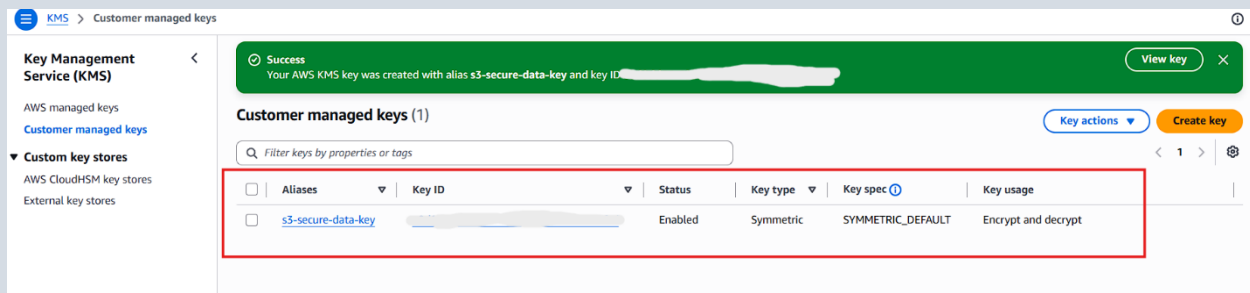


Figure 3. AWS KMS Key Creation and Administrative Configuration

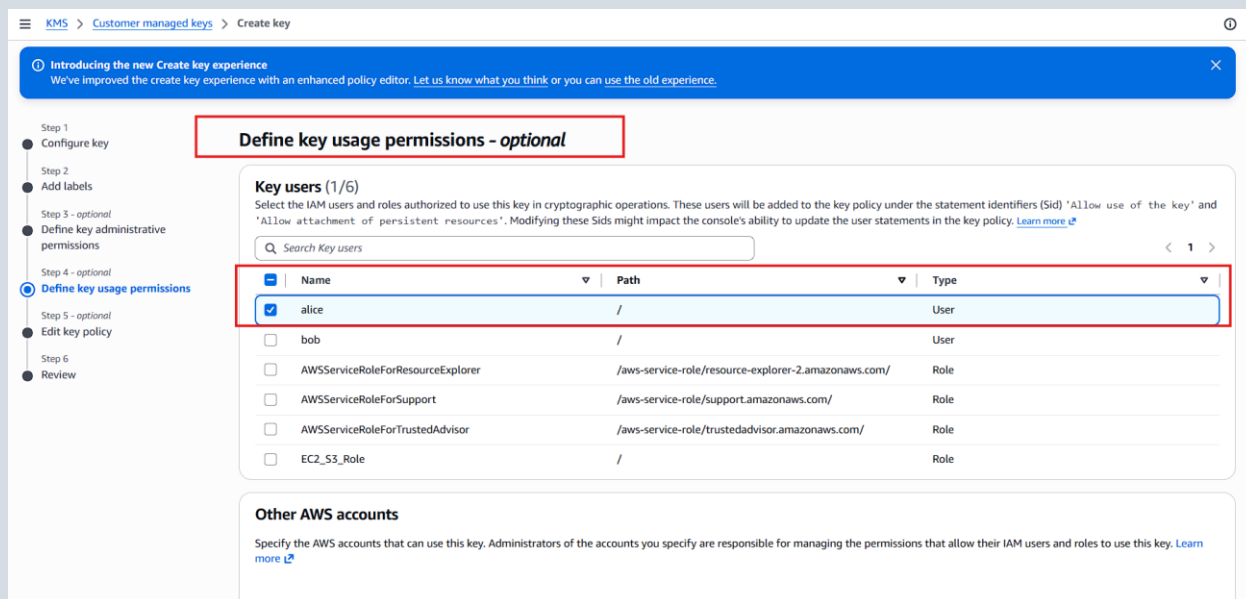


Figure 4. Alice Configured as an Authorized AWS KMS Key User

Explanation

These screenshots demonstrate the complete lifecycle of creating the customer-managed AWS KMS key, configuring administrative permissions, and authorizing Alice to use the encryption key. By granting Alice access to the KMS key, encrypted data stored in Amazon S3 could later be uploaded and downloaded successfully while remaining inaccessible to unauthorized users.

Security Benefit

Customer-managed KMS keys provide greater security and administrative control compared to AWS-managed keys. They enable precise permission management, auditing, and encryption policy enforcement.

Result

The customer-managed AWS KMS key was successfully created and configured for secure object encryption.

Step 3 – Configuring Default Server-Side Encryption Using AWS KMS (SSE-KMS)

Objective

The objective of this step was to configure the Amazon S3 bucket to automatically encrypt every object uploaded into the bucket using Server-Side Encryption with AWS Key Management Service (SSE-KMS). This ensures that all data remains encrypted at rest without requiring manual encryption by the user.

Implementation

After successfully creating the customer-managed AWS KMS key, the suraj-secure-data Amazon S3 bucket was configured to use the newly created KMS key as its default encryption mechanism. Within the bucket properties, Default Encryption was enabled and AWS KMS Keys (SSE-KMS) was selected. The previously created customer-managed KMS key was then assigned as the default encryption key.

With this configuration, every object uploaded into the bucket is automatically encrypted before being written to storage. Users no longer need to specify encryption parameters during upload because Amazon S3 transparently encrypts all objects using the selected KMS key.

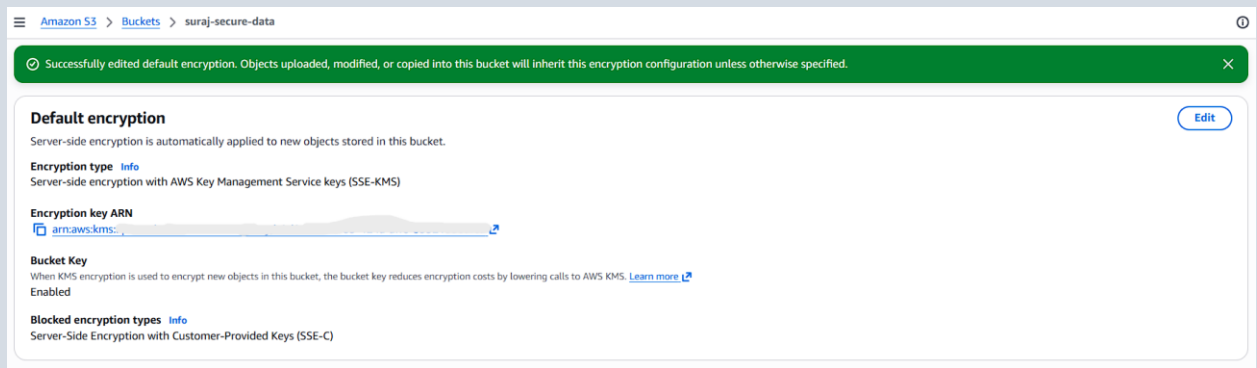


Figure 5. Default Encryption Configured Using Customer-Managed AWS KMS Key

Explanation

The screenshot demonstrates that Server-Side Encryption using AWS KMS (SSE-KMS) has been successfully enabled for the Amazon S3 bucket. The bucket is now configured to automatically encrypt all uploaded objects using the customer-managed encryption key created in the previous step. This configuration protects sensitive information against unauthorized access to the underlying storage infrastructure.

Security Benefit

Enabling default encryption provides the following advantages:

- Automatic encryption for every uploaded object.
- Centralized encryption key management.
- Fine-grained access control through AWS KMS.
- Compliance with cloud security best practices.
- Protection of data at rest.

Result

The Amazon S3 bucket was successfully configured to encrypt all stored objects automatically using the customer-managed AWS KMS key.

Step 4 – Creating a Custom IAM Policy for Authorized Access

Objective

The objective of this step was to create a custom IAM policy that grants only the minimum permissions required for the authorized user (Alice) to work with the encrypted Amazon S3 bucket.

Implementation

A customer-managed IAM policy named Alice-S3-Access-Policy was created. Instead of granting full administrative permissions, the policy followed the Principle of Least Privilege by allowing only the required S3 operations.

The following permissions were included:

- s3:ListAllMyBuckets
- s3:ListBucket
- s3:GetObject
- s3:PutObject

The permissions were restricted to the suraj-secure-data bucket and its objects.

This approach ensured that Alice could:

- View available buckets.
- Open the authorized bucket.
- Upload objects.
- Download objects.

while preventing unnecessary administrative privileges.

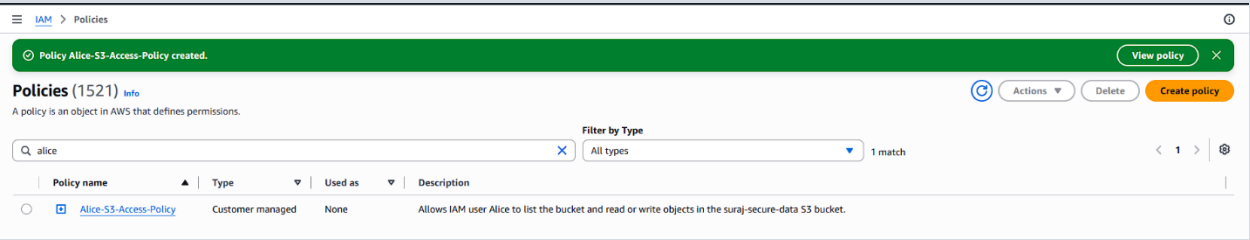


Figure 6. Custom IAM Policy Created

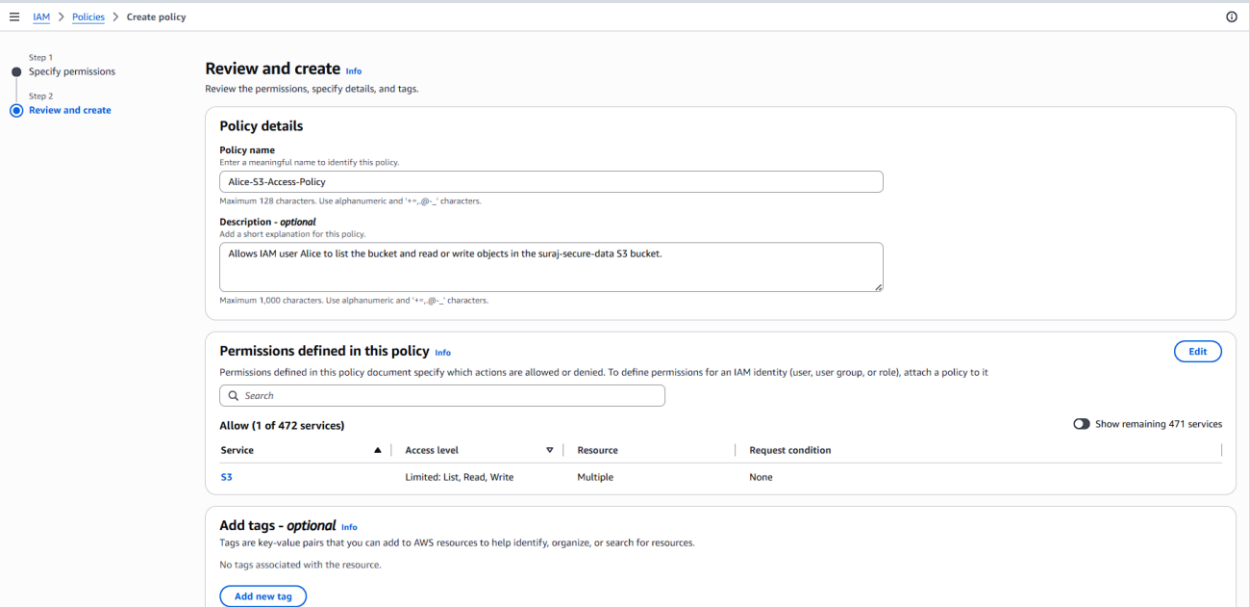


Figure 7. IAM Policy Permission Configuration

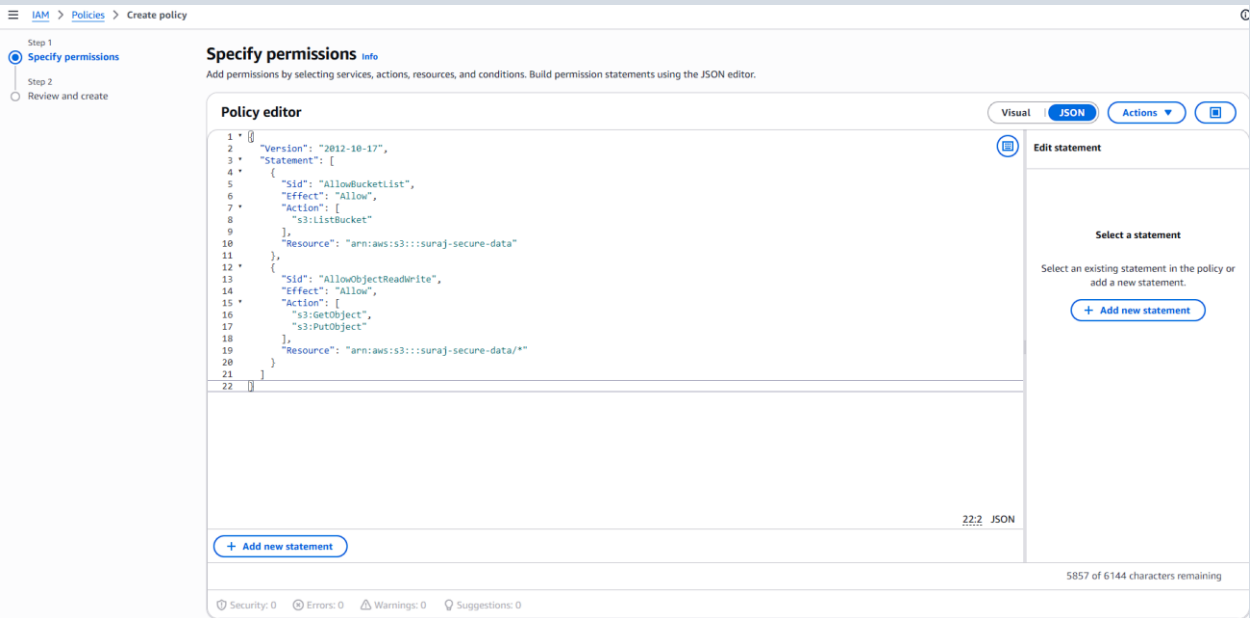


Figure 8. JSON Definition of the Custom IAM Policy

Explanation

These screenshots illustrate the complete creation of the custom IAM policy. The JSON policy explicitly grants only the permissions required for the laboratory exercise. By limiting access to specific Amazon S3 operations and the designated bucket, the implementation reduces the attack surface while maintaining functionality.

Security Benefit

Applying the Principle of Least Privilege minimizes the risk of accidental or malicious misuse of AWS resources. Users receive only the permissions necessary to perform their assigned tasks.

Result

The custom IAM policy was successfully created and prepared for assignment to the authorized IAM user.

Step 5 – Attaching the Custom IAM Policy to Alice

Objective

The objective of this step was to assign the custom IAM policy to the authorized IAM user (Alice) so that she could securely interact with the encrypted Amazon S3 bucket.

Implementation

The previously created Alice-S3-Access-Policy was attached directly to the IAM user Alice. Before applying the policy, AWS displayed a review page summarizing the permissions being granted. After verification, the policy was successfully attached.

Once attached, Alice inherited the permissions defined in the custom policy, enabling secure interaction with the protected bucket while preventing unnecessary administrative access.

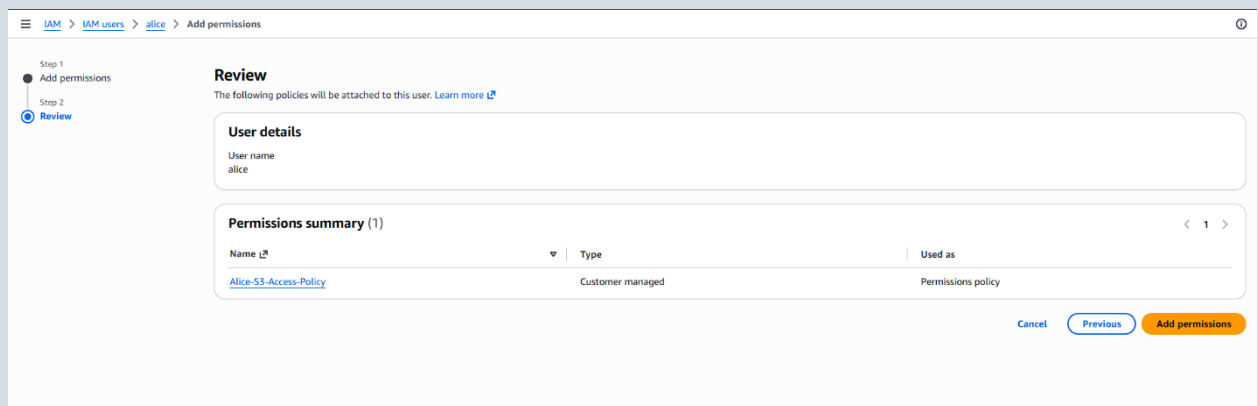


Figure 9. Review Before Attaching the IAM Policy

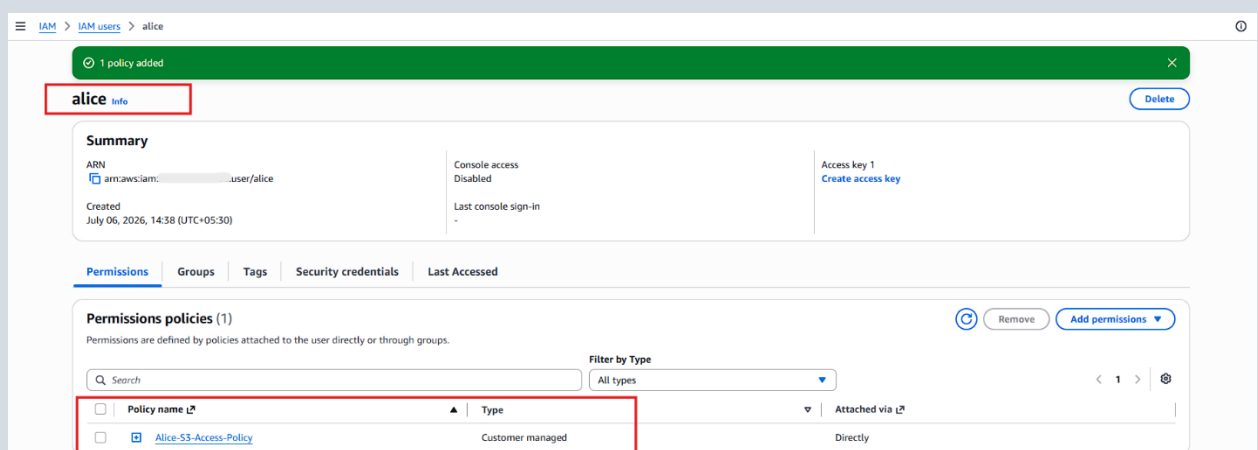


Figure 10. Custom IAM Policy Successfully Attached to Alice

Explanation

The screenshots confirm that the custom IAM policy was successfully assigned to Alice. The AWS IAM console displays the attached policy under Alice's permissions, indicating that the policy is active and will be enforced for all future AWS operations performed by Alice.

Security Benefit

Directly attaching a least-privilege IAM policy ensures that Alice can perform only the intended operations while reducing the risk of privilege escalation or unauthorized administrative actions.

Result

Alice was successfully granted controlled access to the encrypted Amazon S3 bucket.

Step 6 – Enabling AWS Management Console Access

Objective

The objective of this step was to provide both IAM users (Alice and Bob) with AWS Management Console access so that permission validation could be performed through both the graphical interface and the AWS CLI.

Implementation

Console access was enabled separately for Alice and Bob by creating login credentials for each IAM user. Temporary passwords were assigned during user creation, allowing each user to sign in to the AWS Management Console using their respective IAM credentials.

Although both users could successfully authenticate, their available AWS resources depended entirely upon the permissions granted through IAM policies and bucket policies.

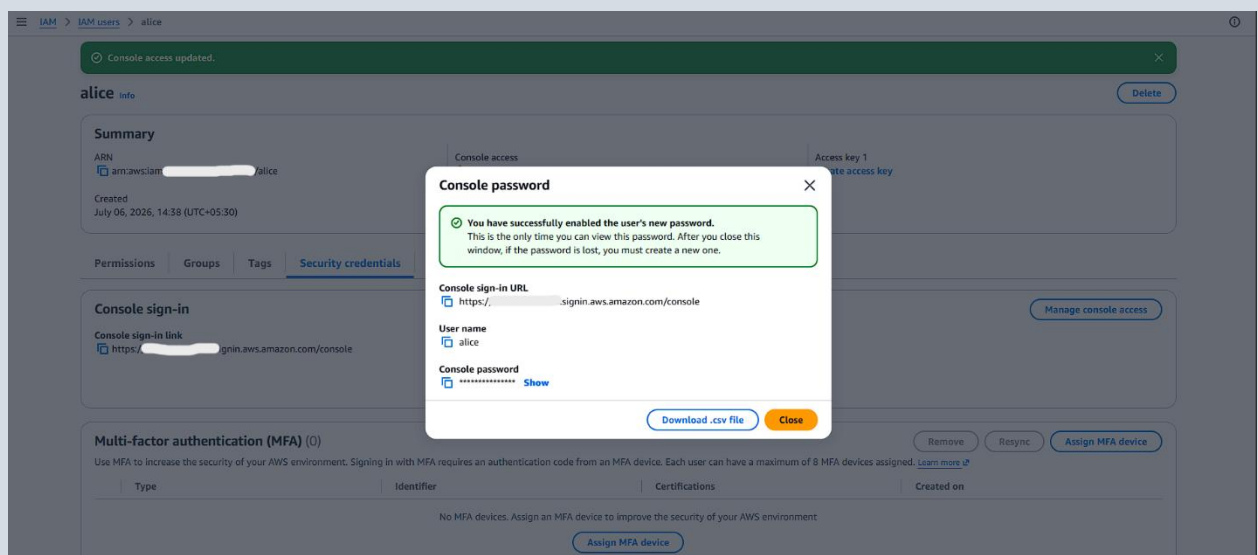


Figure 11. AWS Management Console Access Enabled for Alice

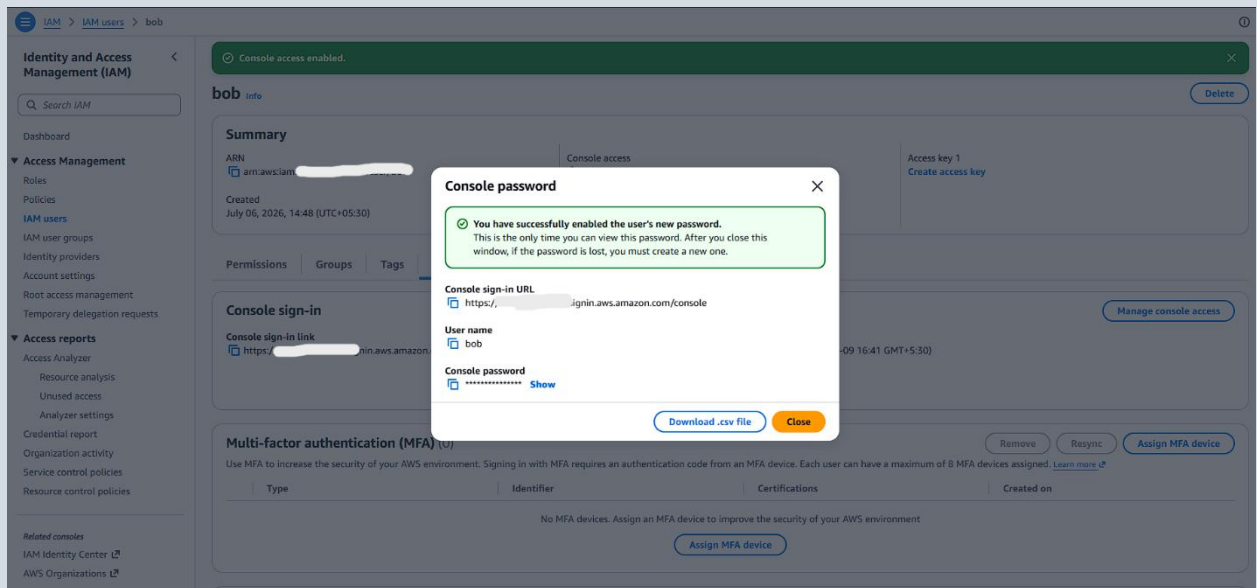


Figure 12. AWS Management Console Access Enabled for Bob

Explanation

These screenshots demonstrate that both IAM users were successfully configured with console login capabilities. This step enabled practical validation of permissions through the AWS Management Console in addition to command-line testing.

Security Benefit

Separating users into individual IAM identities provides accountability, auditing, and independent permission management. Every user action can be traced back to the originating IAM identity.

Result

Both Alice and Bob successfully received AWS Console login access.

Step 7 – Configuring the Amazon S3 Bucket Policy

Objective

The objective of this step was to implement a resource-based security policy that prevents unauthorized identities from accessing objects stored within the encrypted Amazon S3 bucket.

Implementation

A custom bucket policy was created and applied to the suraj-secure-data bucket. The policy used an Explicit Deny statement with a condition that exempted only the authorized IAM user (Alice). Any other IAM principal attempting to upload or download objects was denied access, regardless of any additional IAM permissions they might possess.

After reviewing the JSON policy for correctness, the configuration was successfully saved and applied to the bucket.

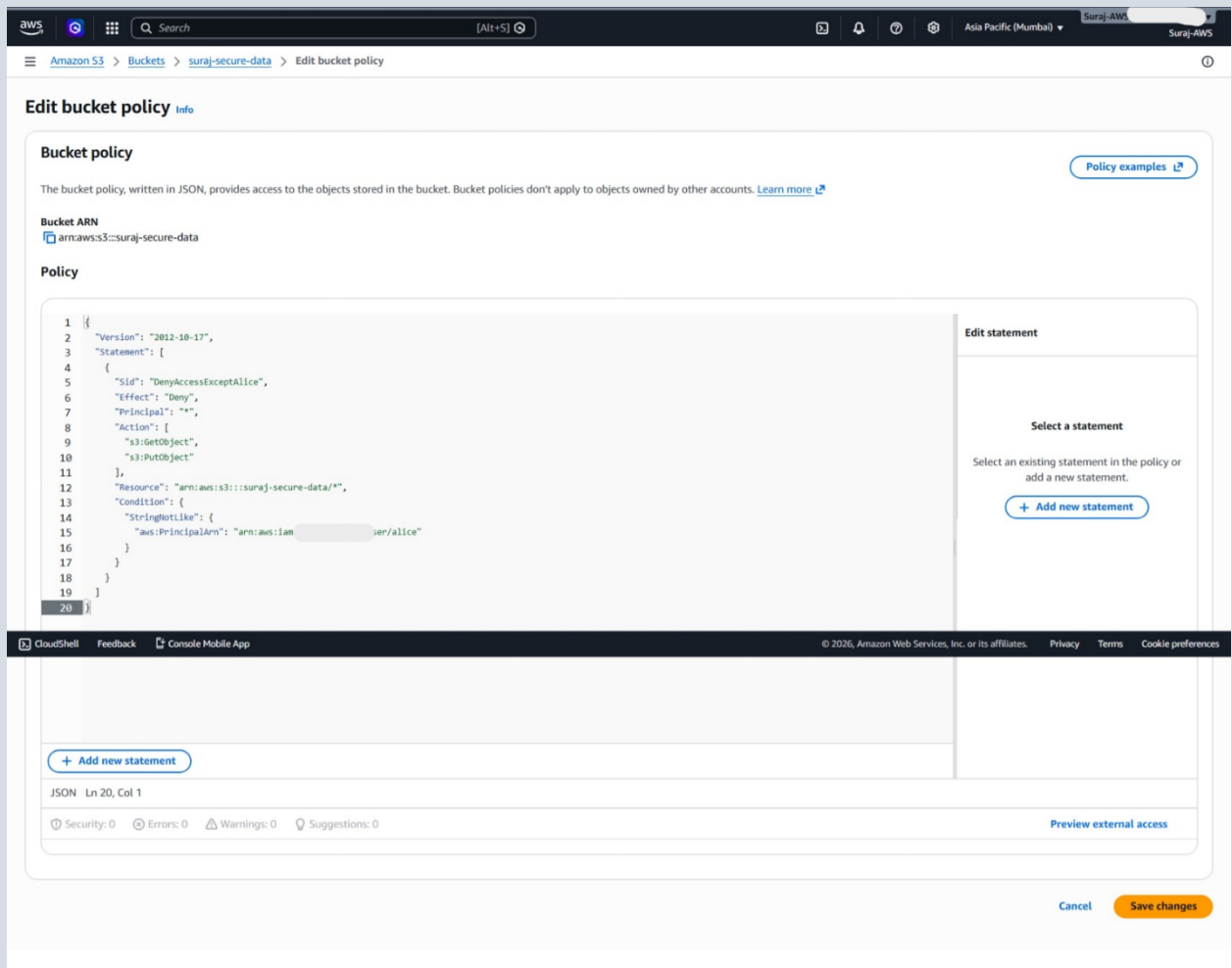


Figure 13. Bucket Policy JSON Before Saving

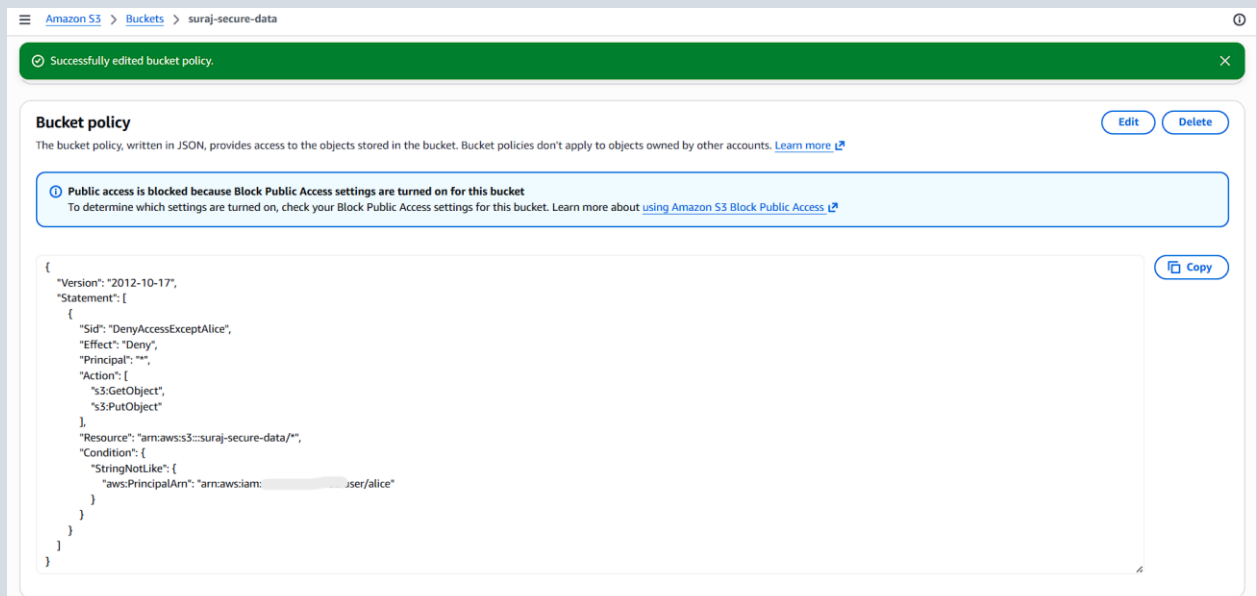


Figure 14. Bucket Policy Successfully Applied

Explanation

The screenshots show the complete bucket policy configuration before and after being applied. The policy explicitly denies GetObject and PutObject operations to all principals except Alice, providing an additional layer

of protection beyond IAM permissions. This demonstrates the use of resource-based policies to strengthen security.

Security Benefit

Bucket policies act as an additional security layer. Even if another IAM user receives broader permissions, the explicit deny in the bucket policy prevents unauthorized access to the protected bucket.

Result

The bucket policy was successfully configured, ensuring that only the authorized IAM user could access objects stored within the encrypted bucket.

Step 8 – Validating Authorized User (Alice) Access

Objective

The objective of this step was to verify that the authorized IAM user Alice could successfully authenticate to AWS and access the encrypted Amazon S3 bucket using both the AWS Management Console and the AWS Command Line Interface (AWS CLI). This validation confirms that the IAM policy, bucket policy, and AWS KMS permissions are correctly configured.

Step 8.1 – Alice's First Login to AWS Console

Implementation

After enabling console access, Alice signed in to the AWS Management Console using her IAM credentials. The login was successful, confirming that the user account and authentication settings were correctly configured. The AWS Region was changed to Asia Pacific (Mumbai) to match the resources created during the lab.

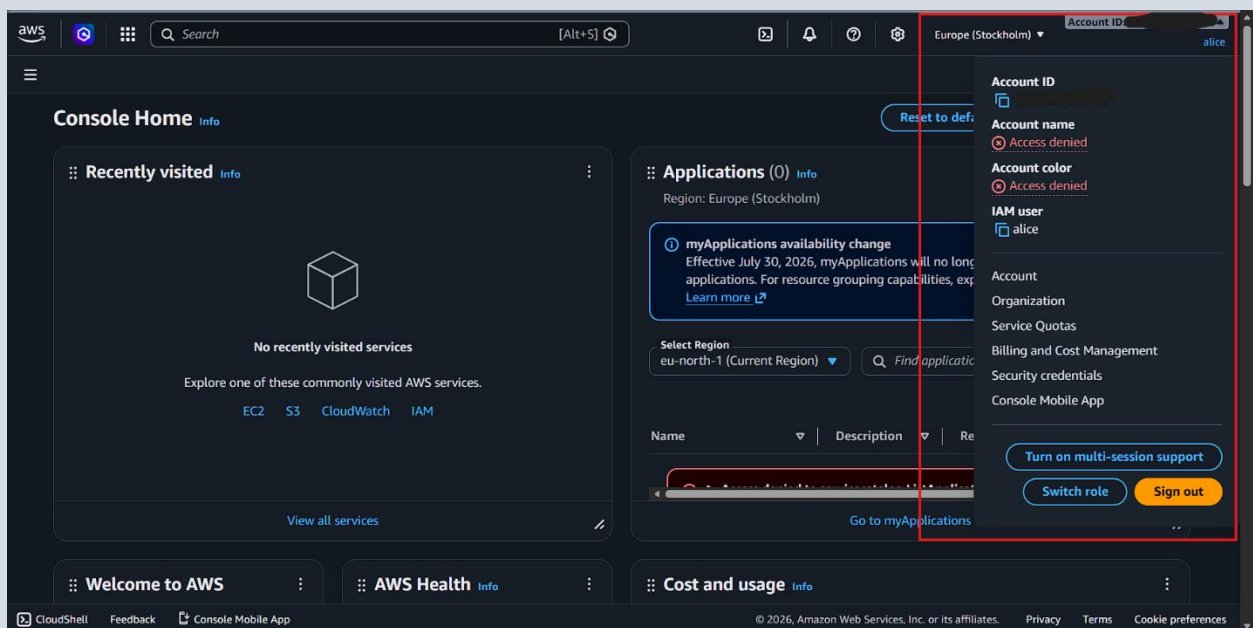


Figure 15. Alice Successfully Logged in to the AWS Management Console

Explanation

This screenshot confirms that Alice successfully authenticated using her IAM username and password. Access to the AWS Console was established without using the root account, demonstrating the recommended AWS security practice of using IAM identities for daily administrative tasks.

Security Benefit

Using IAM users instead of the AWS root account improves accountability, reduces risk, and supports the Principle of Least Privilege.

Result

Alice successfully logged in to the AWS Management Console.

Step 8.2 – Creating Programmatic Access for Alice

Implementation

To allow secure interaction with AWS services through the AWS CLI, an Access Key ID and Secret Access Key were generated for Alice. These credentials were stored securely and later used to configure the AWS CLI.

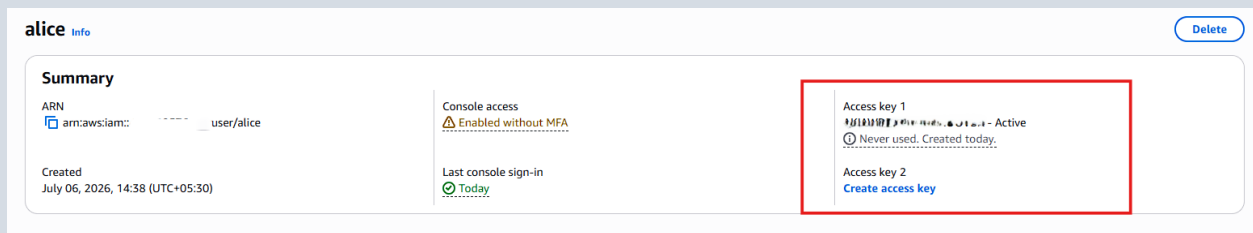


Figure 16. Programmatic Access Enabled for Alice

Explanation

The screenshot shows that an active access key was successfully created for Alice. These credentials allow secure API-based communication with AWS services while maintaining individual identity tracking.

Security Benefit

Programmatic access enables automation and command-line administration while maintaining user-level auditing and access control.

Result

Programmatic access for Alice was successfully configured.

Step 8.3 – Configuring AWS CLI for Alice

Implementation

The AWS CLI was configured using Alice's Access Key ID, Secret Access Key, AWS Region (ap-south-1), and JSON output format. Authentication was verified using:

```
aws sts get-caller-identity
```

A local test file named text.txt was also created to prepare for the upload validation.

```

Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\sssom> aws --version
aws-cli/2.35.20 Python/3.14.6 Windows/11 exe/AMD64
PS C:\Users\sssom> aws sts get-caller-identity
{
  "UserId": "AIDAJD4UWU3U8K3K2R2V2",
  "Account": "000000000000",
  "Arn": "arn:aws:iam::000000000000:user/alice"
}

PS C:\Users\sssom> aws s3 ls
2026-07-09 14:07:05 suraj-secure-data
PS C:\Users\sssom> aws s3 ls s3://suraj-secure-data
PS C:\Users\sssom> cd desktop
PS C:\Users\sssom\desktop> echo "AWS Cloud Security Lab - Alice Upload Test" > text.txt
PS C:\Users\sssom\desktop> dir text.txt

Directory: C:\Users\sssom\desktop


Mode                LastWriteTime         Length Name
----                -
-a-----         10-07-2026   09:51 PM             90 text.txt

PS C:\Users\sssom\desktop> |

```

Figure 17. AWS CLI Successfully Configured for Alice

Explanation

The screenshot demonstrates that Alice's AWS CLI configuration was successful. The `aws sts get-caller-identity` command returned Alice's IAM identity, confirming that subsequent CLI operations would execute using Alice's permissions.

Security Benefit

Verifying the authenticated identity before performing administrative actions prevents accidental execution under the wrong IAM user.

Result

AWS CLI was successfully configured for Alice.

Step 8.4 – Uploading an Object to the Encrypted Bucket

Implementation

Using the AWS CLI, Alice uploaded the previously created `text.txt` file into the encrypted Amazon S3 bucket. The upload completed successfully, confirming that the IAM policy and AWS KMS permissions allowed the operation.

The following command was executed:

```
aws s3 cp text.txt s3://suraj-secure-data
```

The bucket contents were then listed using:

```
aws s3 ls s3://suraj-secure-data
```



```

PS C:\Users\██████\desktop> aws s3 cp text.txt s3://suraj-secure-data
upload: .\text.txt to s3://suraj-secure-data/text.txt
PS C:\Users\██████\desktop> aws s3 ls s3://suraj-secure-data
2026-07-10 21:56:49          90 text.txt
PS C:\Users\██████\desktop> |

```

Figure 18. Successful Object Upload and Bucket Listing Using AWS CLI

Explanation

The screenshot confirms that the upload completed successfully and that the uploaded object appears within the bucket listing. This demonstrates that Alice possesses the required PutObject and ListBucket permissions.

Security Benefit

Only authorized users possessing both IAM permissions and AWS KMS permissions can upload encrypted objects into the bucket.

Result

Object uploaded successfully and verified through the AWS CLI.

Step 8.5 – Viewing Uploaded Objects in AWS Console

Implementation

After uploading the object using the AWS CLI, Alice navigated to the Amazon S3 Console and opened the suraj-secure-data bucket. The uploaded object was successfully displayed.

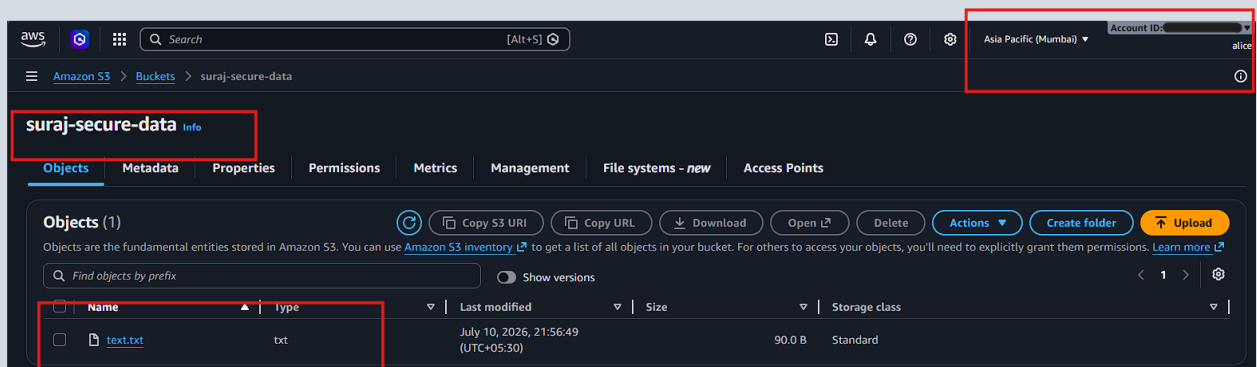


Figure 19. Uploaded Object Successfully Visible in the Amazon S3 Console

Explanation

The screenshot verifies that the uploaded object appears within the Amazon S3 Console. This confirms consistency between CLI operations and the graphical AWS interface.

Security Benefit

Viewing encrypted objects through the AWS Console confirms that permissions are consistently enforced across multiple AWS interfaces.

Result

Uploaded object successfully verified through the AWS Console.

Step 8.6 – Downloading the Encrypted Object

Implementation

To validate read permissions, Alice downloaded the encrypted object using:

```
aws s3 cp s3://suraj-secure-data/text.txt downloaded-text.txt
```

The downloaded file was then opened to verify its contents.

```
PS C:\Users\ [redacted] \desktop> aws s3 cp s3://suraj-secure-data/text.txt downloaded-text.txt
download: s3://suraj-secure-data/text.txt to .\downloaded-text.txt
PS C:\Users\ [redacted] \desktop> type downloaded-text.txt
AWS Cloud Security Lab - Alice Upload Test
PS C:\Users\ [redacted] \desktop> |
```

Figure 20. Successful Download and Verification of Encrypted Object

Explanation

The screenshot confirms that Alice successfully downloaded the encrypted object and verified its contents. AWS KMS automatically decrypted the object because Alice possessed the required KMS permissions.

Security Benefit

This demonstrates that encryption remains transparent to authorized users while continuing to protect the stored data.

Result

Encrypted object successfully downloaded and verified.

Step 9 – Validating Unauthorized User (Bob) Access

Objective

The objective of this phase was to demonstrate that an unauthorized IAM user (Bob) could not access encrypted resources protected by IAM policies, bucket policies, and AWS KMS permissions.

Step 9.1 – Bob Login Verification

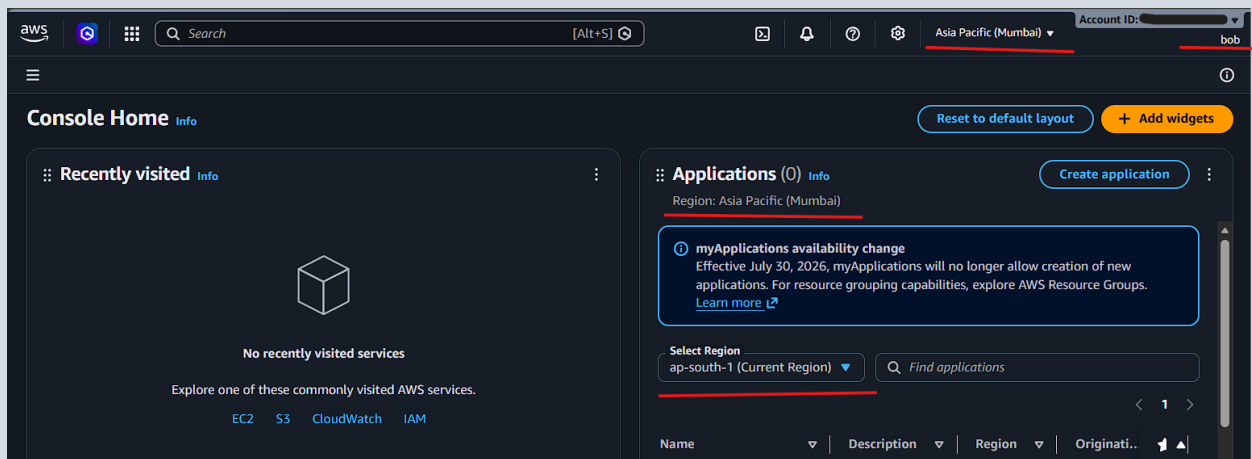


Figure 21. Bob Successfully Logged in to AWS Console

Explanation

Bob successfully authenticated to the AWS Console, confirming that login credentials were valid. However, authentication alone did not grant access to protected resources.

Step 9.2 – AWS CLI Configuration for Bob

[illegible]

Figure 22. AWS CLI Successfully Configured for Bob

Explanation

Bob's CLI credentials were configured and verified using `aws sts get-caller-identity`, confirming that subsequent operations were executed using Bob's IAM identity.

Step 9.3 – Console Access Denied

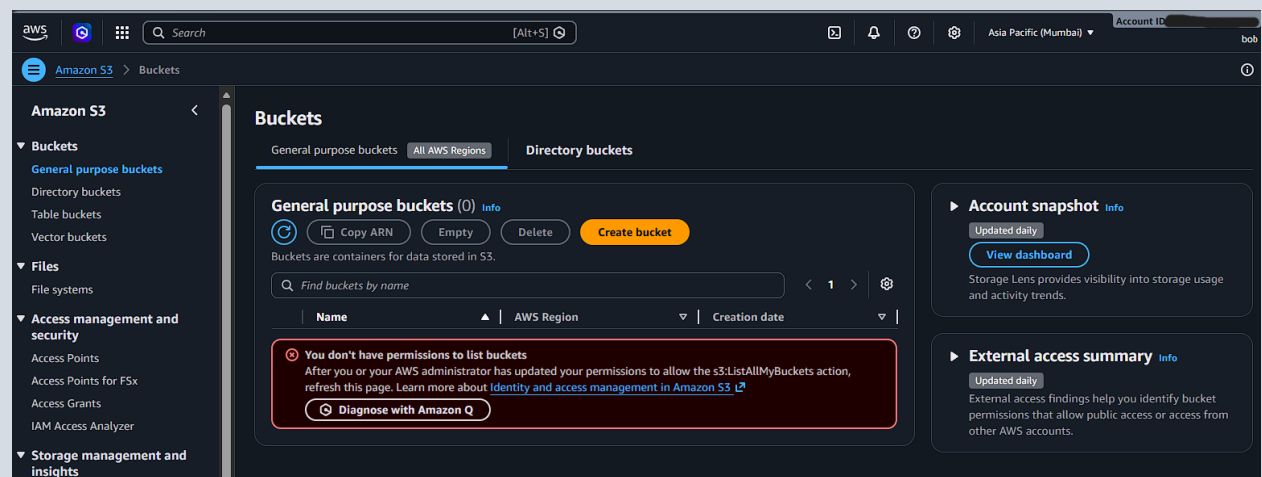


Figure 23. Bob Denied Access to Amazon S3 Buckets

Explanation

Bob was unable to list Amazon S3 buckets because he lacked the `s3:ListAllMyBuckets` permission. This confirmed that the IAM policy correctly restricted bucket discovery.

Step 9.4 – Direct Bucket Access Denied

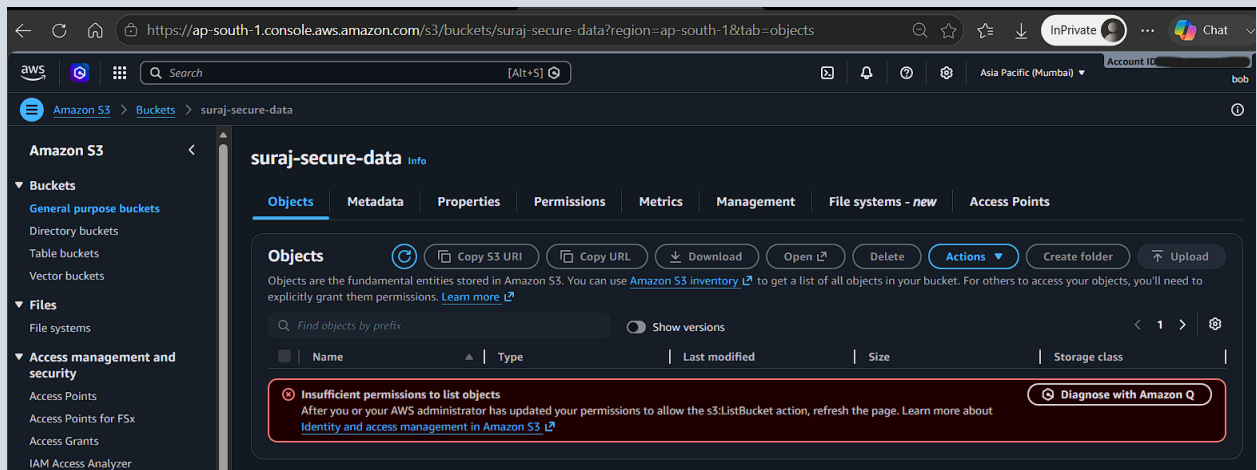


Figure 24. Bob Unable to View Objects Inside the Bucket

Explanation

Although Bob navigated directly to the bucket URL, the AWS Console displayed Insufficient permissions to list objects. This demonstrated that bucket-level permissions prevented unauthorized object enumeration.

Step 9.5 – CLI Bucket Listing Denied

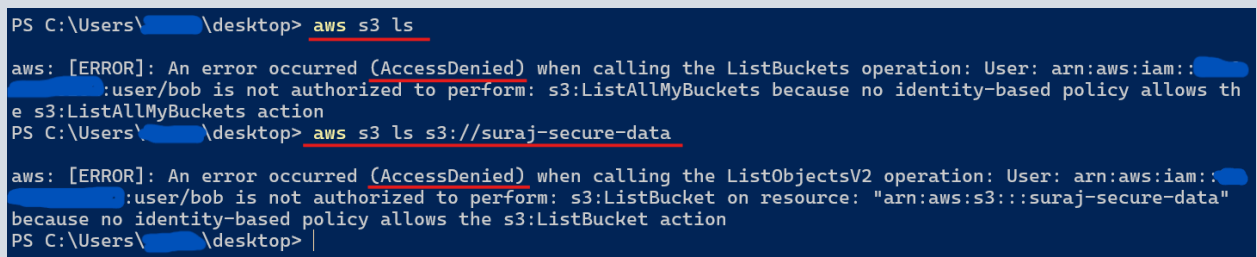


Figure 25. AWS CLI Bucket Listing Denied for Bob

Explanation

Executing `aws s3 ls` and `aws s3 ls s3://suraj-secure-data` resulted in AccessDenied errors. Bob lacked both `s3:ListAllMyBuckets` and `s3:ListBucket` permissions.

Step 9.6 – Upload Denied by AWS KMS

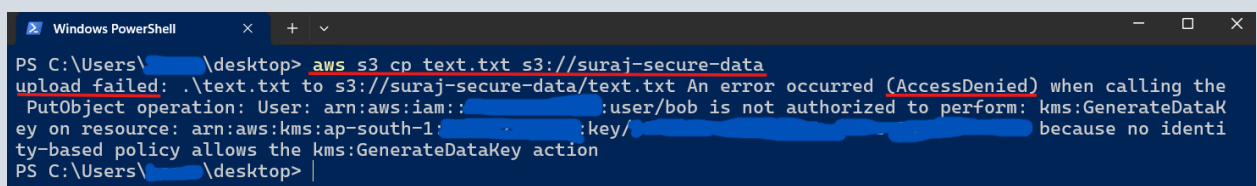


Figure 26. Unauthorized Upload Blocked by AWS KMS

Explanation

Bob attempted to upload an object to the encrypted bucket. The upload failed because AWS KMS denied the `kms:GenerateDataKey` permission. This demonstrates that both IAM permissions and KMS authorization are required to upload objects into an SSE-KMS protected bucket.

Step 9.7 – Download Denied

```
PS C:\Users\██████\desktop> aws s3 cp s3://suraj-secure-data/text.txt bob-download.txt
fatal error: An error occurred (403) when calling the HeadObject operation: Forbidden
PS C:\Users\██████\desktop> |
```

Figure 27. Unauthorized Download Attempt Blocked

Explanation

Bob attempted to download an encrypted object using the AWS CLI. The operation failed with a 403 Forbidden error because he lacked the required permissions to access or decrypt the object.

8. Security Validation

The practical implementation successfully demonstrated a layered cloud security architecture. Amazon S3 encryption using a customer-managed AWS KMS key protected all objects stored in the bucket, while IAM policies granted only the minimum permissions required by the authorized user. The bucket policy introduced an additional layer of protection by explicitly denying access to unauthorized principals.

Testing performed using both the AWS Management Console and AWS CLI confirmed that Alice could successfully upload, list, and download encrypted objects. Conversely, Bob was prevented from listing buckets, viewing objects, uploading files, and downloading encrypted content. The observed AccessDenied, kms:GenerateDataKey, and 403 Forbidden errors verified that AWS security controls were functioning as intended.

9. Challenges Encountered and Troubleshooting

During the implementation, several permission-related challenges were encountered and resolved:

- Alice initially could not view the S3 bucket because the IAM policy lacked the s3:ListAllMyBuckets permission. This permission was added to allow bucket visibility in both the AWS Console and AWS CLI.
- AWS CLI identities were verified using `aws sts get-caller-identity` before performing operations to ensure commands were executed using the correct IAM user.
- Bob's upload attempt failed due to the missing kms:GenerateDataKey permission, demonstrating the interaction between Amazon S3 and AWS KMS.
- Bob's download attempt returned a 403 Forbidden error, confirming that unauthorized users could not retrieve encrypted objects.
- Region settings were standardized to Asia Pacific (Mumbai) to ensure all AWS resources were accessed consistently.

10. Security Best Practices Implemented

The following AWS security best practices were implemented throughout this laboratory:

- Customer-managed AWS KMS keys were used instead of AWS-managed keys to provide greater administrative control.
- Default server-side encryption (SSE-KMS) was enabled to protect all objects stored in Amazon S3.
- Separate IAM users were created for authorized and unauthorized access testing.
- Least-privilege IAM policies were applied to restrict permissions to only the required actions.
- Bucket policies were configured with explicit deny rules to prevent unauthorized object access.
- AWS CLI authentication was verified before performing administrative operations.
- Access keys were generated only for programmatic access and handled securely.
- The AWS root account was not used for routine administration.

11. Learning Outcomes

Through this laboratory exercise, the following practical skills were developed:

- Creation and configuration of Amazon S3 buckets.
- Management of customer-managed AWS KMS encryption keys.
- Implementation of Server-Side Encryption using AWS KMS (SSE-KMS).
- Development of custom IAM policies following the Principle of Least Privilege.
- Configuration of AWS CLI for secure programmatic access.
- Application of Amazon S3 bucket policies for resource-level protection.
- Validation of authorized and unauthorized access using both the AWS Management Console and AWS CLI.
- Troubleshooting AWS permission-related errors involving IAM, S3, and KMS.
- Understanding the interaction between encryption, identity management, and resource-based security controls.

12. Conclusion

This laboratory successfully demonstrated how multiple AWS security services can be integrated to protect sensitive cloud storage. By combining Amazon S3, AWS Identity and Access Management (IAM), AWS Key Management Service (AWS KMS), bucket policies, and the AWS Command Line Interface (AWS CLI), a secure storage environment was implemented that follows industry best practices.

The authorized user (Alice) successfully authenticated, uploaded, listed, and downloaded encrypted objects, confirming that the configured IAM policy and AWS KMS permissions functioned correctly. The unauthorized user (Bob) was consistently denied access through multiple independent security mechanisms, including IAM permissions, bucket policies, and AWS KMS authorization. These results demonstrate the effectiveness of a defense-in-depth approach, where multiple layers of security work together to protect cloud resources.

Overall, this lab provided valuable hands-on experience in implementing encryption at rest, least-privilege access control, and practical validation of secure cloud storage using Amazon Web Services. It reinforced the importance of combining identity management, resource-based policies, and encryption to safeguard sensitive organizational data in modern cloud environments.

13. Deliverable: Short Report

How KMS Encryption Protects the Data

Server-Side Encryption with AWS KMS (SSE-KMS) ensures that every object written to the suraj-secure-data bucket is automatically encrypted at rest using a customer-managed KMS key, so data is never stored in plaintext on the underlying S3 infrastructure. Because the key is customer-managed rather than AWS-managed, the account owner retains full administrative control over who can use the key to encrypt or decrypt objects, independent of the S3 bucket permissions themselves. This creates a second, cryptographic layer of protection: even if a user somehow bypassed IAM or bucket-level permissions, they would still need explicit KMS key permissions (such as `kms:GenerateDataKey` and `kms:Decrypt`) to read or write object data. This was demonstrated in the lab when Bob's upload attempt failed specifically because AWS KMS denied the `kms:GenerateDataKey` permission, confirming that encryption enforcement operates independently of, and in addition to, standard access controls.

How the Applied Access Control Prevents Unauthorized Access

Access to the encrypted bucket was restricted using a combination of least-privilege IAM policies, an explicit-deny bucket policy, and KMS key permissions, so that no single control point had to work alone. The custom IAM policy (Alice-S3-Access-Policy) granted Alice only the specific permissions required (`s3:ListAllMyBuckets`, `s3:ListBucket`, `s3:GetObject`, `s3:PutObject`) scoped to the suraj-secure-data bucket, while the bucket policy added an explicit deny for every principal except Alice, and the KMS key policy authorized only Alice as a key user. Because these layers reinforce one another, Bob's authentication as a valid IAM user was not enough to grant him access: he was blocked at multiple independent points, including `AccessDenied` errors when listing or reading the bucket, denial of the `kms:GenerateDataKey` permission on upload, and a 403

Forbidden error on download. This defense-in-depth approach means that even a misconfiguration in one layer, such as an overly broad IAM policy, would still be caught by the bucket policy or KMS permissions, ensuring that only explicitly authorized identities can read or write data in the encrypted bucket.

14. References

1. Amazon Web Services. Amazon S3 User Guide.
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
2. Amazon Web Services. AWS Key Management Service (KMS) Developer Guide.
<https://docs.aws.amazon.com/kms/latest/developerguide/>
3. Amazon Web Services. AWS Identity and Access Management (IAM) User Guide.
<https://docs.aws.amazon.com/IAM/latest/UserGuide/>
4. Amazon Web Services. AWS Command Line Interface User Guide.
<https://docs.aws.amazon.com/cli/latest/userguide/>